CSCE 355: JFLAP3 — Building DFAs

Overview

In this assignment you will build seven deterministic finite automata (DFAs) in JFLAP. Each is checked automatically. The assignment is worth **40 points**: Problems 1–6 are worth 5 points each and Problem 7 is worth 10 points.

Getting started

- Read the JFLAP tutorial: `https://jflap.org/tutorial/`.
- JFLAP is a free `.jar` file from `https://www.jflap.org`. You need Java 8 or higher. Start it with `java -jar JFLAP7_1.jar` (use the name of the file you downloaded).
- Unzip this assignment folder. (It is a good idea to append your username to the folder name, e.g. `JFLAP3_jsmith`.)
- In JFLAP use **File** → **Open** to open one of the provided `.jff` files. Each one is an empty *Finite Automaton*. Build your DFA, then **File** → **Save** it *under the same file name*. Do not change the machine type.
- JFLAP shows the empty string as λ. Just leave the empty string as λ; do not change it to ϵ.

#	Your DFA (edit this file)	Test strings (do not edit)	Points
1	`problem1_01then10.jff`	`problem1_01then10_tests.txt`	5
2	`problem2_odd1s_0to1.jff`	`problem2_odd1s_0to1_tests.txt`	5
3	`problem3_no010.jff`	`problem3_no010_tests.txt`	5
4	`problem4_div7.jff`	`problem4_div7_tests.txt`	5
5	`problem5_parens.jff`	`problem5_parens_tests.txt`	5
6	`problem6_01_011.jff`	`problem6_01_011_tests.txt`	5
7	`problem7_01_110_011.jff`	`problem7_01_110_011_tests.txt`	10

What counts as a DFA

Every machine you submit must be a DFA. If a machine is not a DFA it receives **zero credit** for that problem, no matter which strings it happens to get right. This is checked automatically, before any test string is run. A machine is a DFA when:

- each state has **exactly one** outgoing transition on *each* symbol of the problem's alphabet: never two (two arrows leaving a state on the same symbol, even to different states, is nondeterminism) and never zero (a missing arrow is not allowed);
- there are **no λ (empty-string) transitions**;

- every transition reads **exactly one symbol**, and that symbol belongs to the problem's alphabet (a single arrow may carry both symbols, e.g. labeled `0,1`, but a label such as `01` is *not* allowed);
- there is exactly one initial state.

You must draw the trap (dead) state. JFLAP lets you leave transitions out and simply rejects a string that gets stuck, but for this assignment that is *not* a DFA: every state needs an arrow for every symbol. Wherever a string should be rejected for good, send the missing arrows to a non-final trap state that loops to itself on every symbol. A DFA missing any transition receives zero credit for that problem, and the report lists the states and symbols that are missing.

How to check in JFLAP

Test → Highlight Nondeterminism. With your machine open in the editor, choose this menu item. JFLAP highlights every state that has nondeterministic behavior. If nothing is highlighted your machine passes this check; if a state is highlighted, look at its outgoing arrows and fix the conflict. Also make sure you did not leave any λ transition behind. When you single-step a string with **Input → Step**, a DFA has exactly one current state at every point. JFLAP does *not* warn you about missing transitions, so also check by eye that every state has an outgoing arrow for each symbol (for the alphabet $\{0, 1\}$: two arrows per state; the trap state counts as a state). Running `python3 test_dfa.py` (below) checks this too.

Problems

Problem 1 (5 points)

Construct a DFA that recognizes any number of `01`, followed by `10`s, without overlap (e.g. `010` is not in the language recognized by the DFA). The alphabet is $\{0, 1\}$.

- The test file has 8 strings: the first 3 should accept, the last 5 should reject.
- The empty string should accept (see "Testing the empty string" below).

Problem 2 (5 points)

Construct a DFA in 8 or fewer states (I am fine if you delete unreachable states) that is the product construction of two DFAs: one that recognizes an odd number of `1`s, and one that recognizes a binary number with a leading `0` and ending `1`. A string is accepted only if it satisfies *both* conditions. The alphabet is $\{0, 1\}$.

- The test file has 9 strings: the first 4 should accept, the last 5 should reject.

Problem 3 (5 points)

Construct a DFA that recognizes strings without `010` as a substring, or that are just `01` repeated (`01`, `0101`, `010101`, ...). The empty string should be accepted. The alphabet is $\{0, 1\}$.

- The test file has 8 strings: the first 4 should accept, the last 4 should reject.

Problem 4 (5 points)

Construct a DFA to recognize unsigned binary strings evenly divisible by 7. The alphabet is $\{0, 1\}$.

- You may accept the empty string (it will not be tested).
- Leading zeros have no meaning (this is not octal notation): `0101` and `0000101` are both the number 5.
- The test file has 23 strings: the last five should accept, all previous ones reject.

Problem 5 (5 points)

Construct a DFA to recognize, for the alphabet of just parentheses $\Sigma = \{(,)\}$, those strings that have balanced, proper parentheses pairs, with no more than nesting depth 4. For example, you do not have to recognize `(((((())))))` as being in the language (you should reject it), but you do have to recognize `((((()))))` or even `(((())(((()))(((()))))`.

- You may accept the empty string (it will not be tested).
- The test file has 11 strings: the first 5 should accept, the last 6 should reject.
- Once you are done, ask yourself what is interesting about this, do you think? (Not graded.)

Problem 6 (5 points)

Construct a DFA recognizing the concatenation of any number of `01` and/or `011`, zero or more copies of each in any order. The alphabet is $\{0, 1\}$.

- The test file has 8 strings: the first 4 should accept, the last 4 should reject.
- The empty string should accept.

Problem 7 (10 points)

Construct a DFA recognizing the concatenation of any number of `01`, `110`, and/or `011`, zero or more copies of each in any order (the copies may be interleaved; look at the tests). The alphabet is $\{0, 1\}$.

- The test file has 117 strings: all but the last ten should accept, and the last ten should reject.
- The empty string should accept.

Testing your DFAs

In JFLAP

1. Open your DFA (it needs an initial state to be tested).
2. Choose **Input** → **Multiple Run**.
3. Click **Load Inputs** (at the bottom) and select the problem's test file.
4. Click **Run Inputs** and compare each row with what the problem says should accept and reject.

Testing the empty string

The test files do not contain the empty string. In Multiple Run, put the cursor on the last row and press Enter to add a blank line (an empty input), then run again. The empty string should accept in Problems 1, 3, 6, and 7 (and should reject in Problem 2).

From the command line

This folder contains `test_dfa.py`, the same script that is used to grade you (Python 3 is required). From inside this folder run

```
python3 test_dfa.py
```

to check all seven problems, or `python3 test_dfa.py 3` to check only Problem 3. The report tells you, for each problem: whether the file is really a DFA, which provided test strings are wrong, and your score. Run it as often as you like.

Grading

Each problem earns its points only if *all* of the following hold; there is no partial credit within a problem.

- The machine is a DFA (see above).
- Every string in the problem's test file gets the expected result.
- Your DFA recognizes *exactly* the described language. The provided strings are only examples: the grader compares your DFA with the described language on all strings, so a DFA that passes the samples but is wrong somewhere else earns no credit. When this happens the report shows a shortest string on which your DFA is wrong.
- Problem 2 only: at most 8 states are reachable from the initial state (the trap state counts if it is reachable).

Do not edit the test files or `test_dfa.py`. They are replaced with the original versions before grading, so changing them can only make your own testing misleading.

Submission

1. Open **YOURUSERNAME.txt** and put your university username as the *first line* of the file (only your username, nothing else on that line). This is how your submission is identified.
2. Make sure each of your seven DFAs is saved in its provided `.jff` file. Do not rename any of the provided files.
3. Run `python3 test_dfa.py` one last time to make sure everything is as you expect.
4. Submit this entire folder (or a zip of it) as instructed.